SAE 3.2

Primitive Image Format

Compression et décompression d'images

Auteurs :	Yilmaz Kevin Barhani Adam
Formation :	BUT Informatique 2ème année
Établissement :	IUT de Fontainebleau-Sénart
Date de rendu :	1er mars 2025

Rapport d'avancement — Projet Java

Table des matières

1 Introduction **3**
1.1 Contexte et description du sujet . . . 3
1.2 Ce qu'on a développé . . . 3

2 Fonctionnalités **4**
2.1 Lancement des programmes . . . 4
2.2 Le Convertisseur . . . 4
2.3 Le Visualisateur . . . 5
2.3.1 Image affichée normalement . . . 5
2.3.2 Image plus petite que la fenêtre (centrage) . . . 6
2.3.3 Image plus grande que la fenêtre (déplacement à la souris) . . . 7

3 Structure du programme **8**
3.1 Architecture MVC . . . 8
3.2 Description des classes . . . 8
3.2.1 Package Modèle . . . 8
3.2.2 Package Contrôleur . . . 9
3.2.3 Package Vue . . . 10
3.2.4 Points d'entrée . . . 10
3.3 Nos choix techniques . . . 10
3.3.1 Opérations binaires plutôt que Color() . . . 10
3.3.2 EcritureBinaire et LectureBinaire maison . . . 10
3.3.3 Séparation analyser() et ecrire() . . . 11
3.3.4 Représentation des codes en String . . . 11
3.3.5 Convertisseur et Visualisateur hors MVC . . . 11

4 L'arbre binaire de Huffman **12**
4.1 L'arbre de Huffman . . . 12
4.1.1 Les classes Noeud et Feuille . . . 12
4.1.2 Construction de l'arbre . . . 12
4.1.3 Lecture de l'arbre et génération des codes . . . 13

5 Encodage — côté Convertisseur **14**
5.1 Les tables de fréquences . . . 14
5.2 Les codes initiaux de Huffman . . . 14
5.3 Les codes canoniques . . . 14
5.3.1 Pourquoi transformer les codes . . . 14
5.3.2 Comment fonctionne la transformation . . . 15
5.3.3 Ce qu'on stocke dans le fichier . . . 15
5.4 Le format du fichier PIF . . . 15

6 Décodage — côté Visualisateur **16**
6.1 Lecture de l'en-tête 16
6.2 Reconstruction des codes canoniques 16
6.3 Reconstruction de l'arbre 16
6.4 Décodage des pixels 16

7 Conclusion **17**
7.1 Conclusion de Kevin Yilmaz 17
7.2 Conclusion de Adam Barhani 17

1. Introduction

1.1 Contexte et description du sujet

Dans le cadre de la SAE 3.2, on nous a demandé de développer deux programmes Java capables de manipuler un nouveau format de fichier image : le format **PIF** (Primitive Image Format).

Le problème de départ est simple : les formats d'images non compressés prennent beaucoup de place. Une image de 1000x1000 pixels en RGB occupe environ 3 Mo sur le disque, ce qui devient vite problématique. Le format PIF apporte une solution en compressant les données de l'image sans aucune perte de qualité, c'est-à-dire que l'image décompressée est pixel par pixel identique à l'originale.

Pour y arriver, on utilise l'algorithme de **Huffman**, qui consiste à attribuer des codes binaires courts aux couleurs les plus fréquentes et des codes plus longs aux couleurs rares. Le gain de place dépend donc de la distribution des couleurs dans l'image.

1.2 Ce qu'on a développé

On a développé deux programmes distincts :

— **Le Convertisseur** : permet d'ouvrir une image (JPG, PNG...) et de la sauvegarder au format PIF. Il affiche également les tables de fréquences et de codes pour chaque composante RGB.

— **Le Visualisateur** : permet d'ouvrir un fichier PIF et d'afficher l'image qu'il contient dans une fenêtre interactive.

2. Fonctionnalités

2.1 Lancement des programmes

Les deux programmes se lancent via des archives JAR générées par le Makefile.

Commande	Description
`make convert`	Compile et genere `convertisseur.jar`
`make visualize`	Compile et genere `visualisateur.jar`
`make all`	Genere les deux archives JAR en une seu
`make runconvert`	Genere et lance le convertisseur sans argu
`make runvisualize`	Genere et lance le visualisateur sans argu
`make javadoc`	Genere la documentation dans `javadoc/`
`make clean`	Supprime les classes compilees
`make cleanjar`	Supprime les archives JAR
`make mrproper`	Supprime les classes et les archives JAR
`make cleanall`	Supprime tout ce qui est genere
`java -jar visualisateur.jar`	Lance sans argument (JFileChooser)
`java -jar visualisateur.jar fichier.pif`	Ouvre directement le fichier
`java -jar convertisseur.jar`	Lance sans argument (JFileChooser)
`java -jar convertisseur.jar image.png`	Ouvre directement l'image
`java -jar convertisseur.jar image.png sortie.pif`	Definit aussi le fichier de sortie

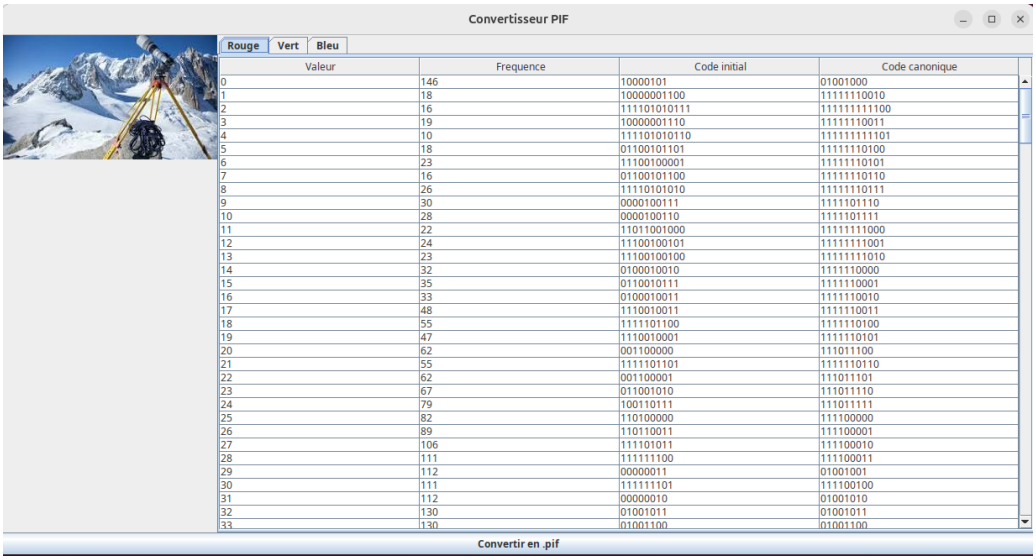
2.2 Le Convertisseur

Le convertisseur peut être lancé avec ou sans argument. Sans argument, un JFileChooser s'ouvre pour sélectionner l'image à convertir. Avec un premier argument, l'image est chargée directement. Un deuxième argument optionnel définit le chemin du fichier de sortie ; sans lui, un JFileChooser s'ouvre au moment de la conversion pour choisir où sauvegarder le fichier PIF.

Une fois l'image chargée, l'utilisateur peut la voir en aperçu dans la partie haute de la fenêtre. Il peut aussi consulter les tables de chaque composante (Rouge, Vert, Bleu) via des onglets : chaque table affiche pour chaque valeur présente dans l'image sa fréquence, son code initial de Huffman et son code canonique. Après avoir consulté ces informations, l'utilisateur peut valider la compression en cliquant sur le bouton "Convertir en .pif".

Si l'image ne peut pas être convertie (par exemple si une composante n'a qu'une seule valeur distincte), le programme refuse la conversion et affiche un message expliquant la

raison du refus.



Valeur	Frequence	Code initial	Code canonique
0	146	10000101	01001000
1	18	10000001100	11111110010
2	16	111101010111	111111111100
3	19	10000001110	11111110011
4	10	111101010110	111111111101
5	18	01100101101	11111110100
6	23	11100100001	11111110101
7	16	01100101100	11111110110
8	26	11110101010	11111110111
9	30	0000100111	11111101110
10	28	0000100110	11111101111
11	22	11011001000	111111111000
12	24	11100100101	111111111001
13	23	11100100100	111111111010
14	32	0100010010	11111110000
15	35	0110010111	11111110001
16	33	0100010011	11111110010
17	48	1110010011	11111110011
18	55	1111101100	11111110100
19	47	1110010001	11111110101
20	62	001100000	111011100
21	55	1111101101	11111110110
22	62	001100001	111011101
23	67	011001010	111011110
24	79	100110111	111011111
25	82	110100000	111100000
26	89	110110011	111100001
27	106	111101011	111100010
28	111	111111100	111100011
29	112	00000011	01001001
30	111	111111101	111100100
31	112	00000010	01001010
32	130	01001011	01001011
33	130	01001100	01001100

FIGURE 2.1 – Interface du Convertisseur avec l'image et les tables de codes

2.3 Le Visualisateur

Le visualisateur peut lui aussi être lancé avec ou sans argument. Sans argument, un JFileChooser filtré sur les fichiers `.pif` s'ouvre pour sélectionner le fichier à afficher. Avec un argument, le fichier est ouvert directement.

La fenêtre s'adapte automatiquement à la taille de l'image affichée, avec une marge de 50 pixels autour de l'image. Si l'image est trop grande pour tenir dans l'écran, la fenêtre prend la taille maximale disponible. La fenêtre est redimensionnable à tout moment.

2.3.1 Image affichée normalement

Quand la taille de l'image est raisonnable, elle s'affiche en entier dans la fenêtre avec la marge prévue autour d'elle.

FIGURE 2.2 – Visualisateur avec une image de taille normale

2.3.2 Image plus petite que la fenêtre (centrage)

Si la fenêtre est agrandie au-delà de la taille de l'image, l'image reste centrée dans la fenêtre. Ce centrage est recalculé automatiquement à chaque redimensionnement de la fenêtre.

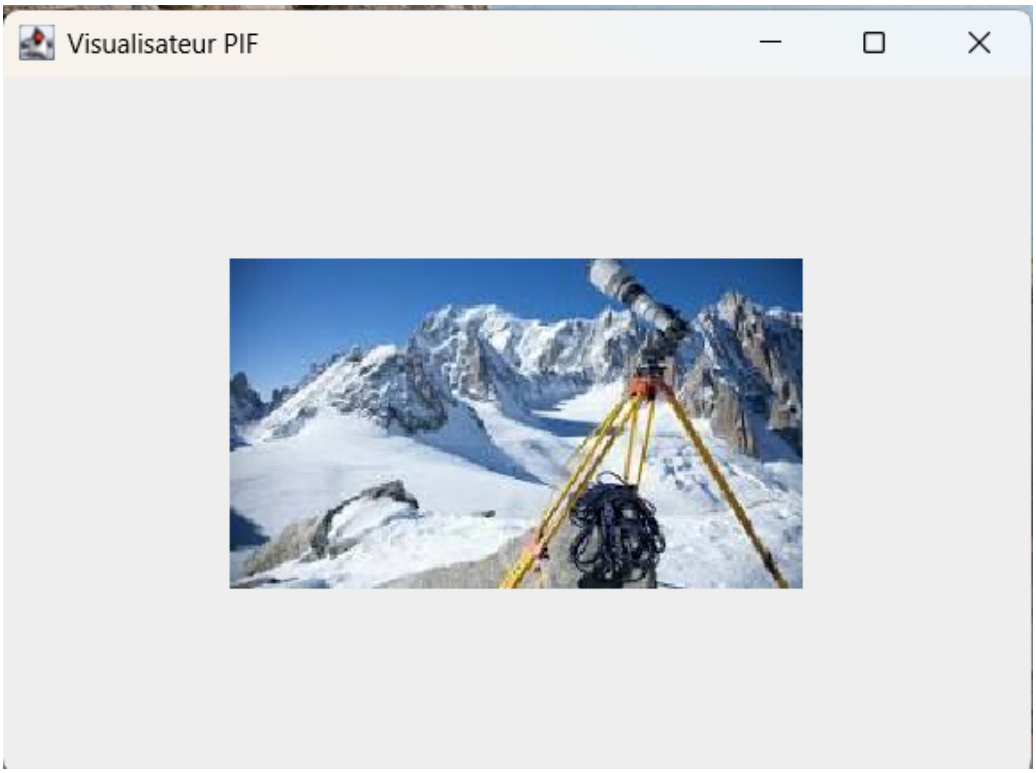


FIGURE 2.3 – Image centrée quand la fenêtre est plus grande que l'image

2.3.3 Image plus grande que la fenêtre (déplacement à la souris)

Si la fenêtre est rétrécie au point qu'elle ne peut plus afficher l'image en entier, seule une portion de l'image est visible. L'utilisateur peut alors déplacer la partie visible en maintenant le clic gauche de la souris enfoncé et en faisant glisser. Le déplacement est fluide et fonctionne dans toutes les directions.

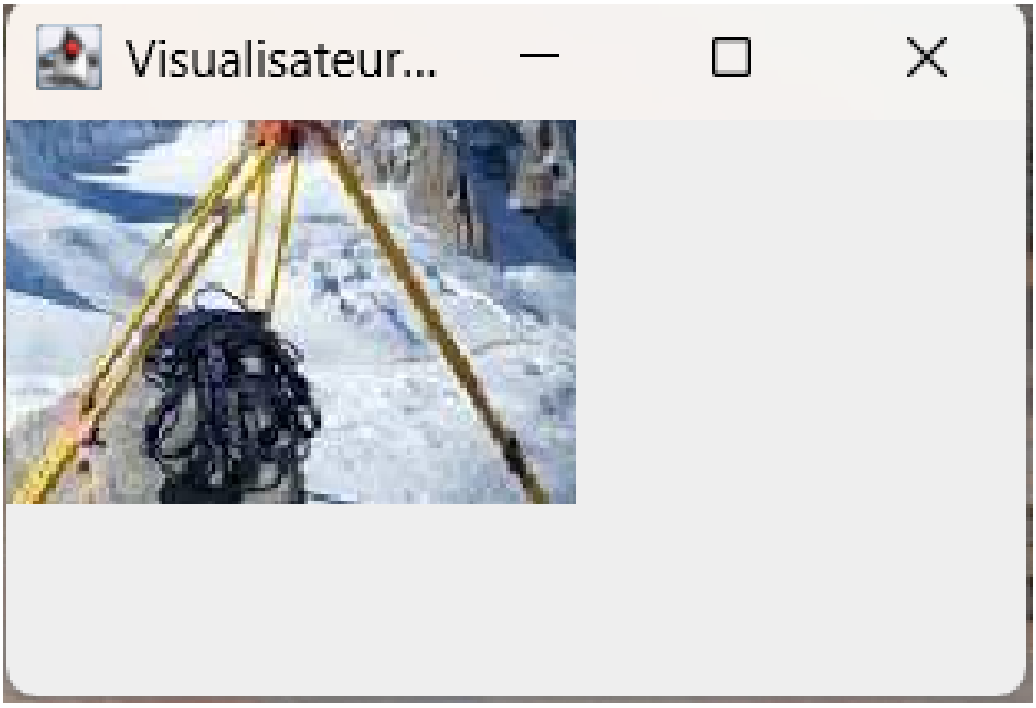


FIGURE 2.4 – Déplacement de la portion visible à la souris

3. Structure du programme

3.1 Architecture MVC

On a choisi d'organiser le projet selon l'architecture **Modèle-Vue-Contrôleur** (MVC). Ce choix est naturel pour ce type de projet car les responsabilités sont clairement séparées :

— Le **Modèle** contient toute la logique métier : les structures de données (arbre, codes, fréquences) et les opérations sur les fichiers binaires.

— La **Vue** gère uniquement l'affichage : les fenêtres, les panneaux, les tables.

— Le **Contrôleur** fait le lien entre les deux : il réagit aux actions de l'utilisateur et appelle le modèle en conséquence.

Les classes `Convertisseur` et `Visualisateur` (les points d'entrée avec les `main`) ont été placées directement dans le package principal, en dehors du MVC, car elles ne font que démarrer le programme en quelques lignes.

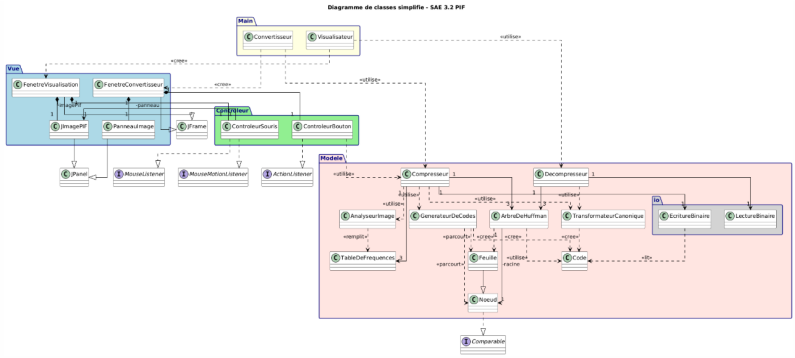


FIGURE 3.1 – Diagramme de classes simplifié du projet

3.2 Description des classes

3.2.1 Package Modèle

`Noeud` Représente un noeud interne de l'arbre de Huffman. Il contient une fréquence, un fils gauche et un fils droit. Il est comparable pour pouvoir être utilisé dans une `PriorityQueue`.

`Feuille` Hérite de `Noeud`. Représente une feuille de l'arbre, c'est-à-dire une valeur de couleur réelle (0 à 255). Elle redéfinit la méthode `estUneFeuille()` qui retourne `true`.

`Code`
: Stocke un code binaire sous forme de `String` et expose des méthodes pour lire bit par bit. On a choisi de garder la représentation en `String` pour sa simplicité de manipulation et de débogage, même si une représentation en tableau de booléens serait plus efficace en mémoire.

`TableDeFrequences`
: Contient un tableau de 256 entiers représentant la fréquence de chaque valeur (0 à 255) pour une composante de couleur.

`AnalyseurImage`
: Parcourt les pixels de l'image et remplit trois `TableDeFrequences` (rouge, vert, bleu). On utilise des opérations binaires (bitwise) plutôt que `Color.getRed()` pour éviter la création d'objets inutiles et améliorer les performances.

`ArbreDeHuffman`
: Construit l'arbre de Huffman à partir d'un tableau de fréquences via une `PriorityQueue`. Contient aussi une méthode `ajouterCode()` utilisée lors de la décompression pour reconstruire l'arbre depuis les codes canoniques.

`GenerateurDeCodes`
: Parcourt l'arbre de Huffman en profondeur et génère un dictionnaire associant chaque valeur à son code initial.

`TransformateurCanonique`
: Transforme un dictionnaire de codes initiaux en codes canoniques. Contient aussi la méthode inverse `reconstruireDepuisLongueurs()` utilisée par le décompresseur.

`EcritureBinaire`
: Permet d'écrire dans un fichier bit par bit, en accumulant les bits dans un tampon et en écrivant octet par octet. Expose des méthodes `ecrireOctet()`, `ecrireShort()` et `ecrireCode()`.

`LectureBinaire`
: Miroir de `EcritureBinaire`. Permet de lire un fichier bit par bit avec un tampon. Expose des méthodes `lireOctet()`, `lireShort()` et `lireBit()`.

3.2.2 Package Contrôleur

`Compresseur`
: Orchestre toute la logique de compression. Expose les méthodes `analyser()` qui calcule les fréquences et génère les codes, et `ecrire()` qui produit le fichier PIF. Les résultats intermédiaires sont stockés en attributs et accessibles via des getters pour l'affichage dans l'interface.

`Decompresseur`
: Orchestre la décompression. Expose la méthode `decompresserEnMemoire()` qui retourne une `BufferedImage` directement sans écrire de fichier intermédiaire.

`ControleurSouris`
: Implémente `MouseListener` et `MouseMotionListener`. Gère le déplacement de l'image dans le Visualisateur en calculant le delta entre la position actuelle et la dernière position enregistrée de la souris.

`ControleurBouton`
: Implémente `ActionListener`. Déclenche la conversion quand l'utilisateur clique sur le bouton, gère le JFileChooser si nécessaire et affiche le résultat.

3.2.3 Package Vue

`JImagePIF`
: Hérite de `JPanel`. Redéfinit `paintComponent()` pour afficher l'image centrée ou décalée selon l'offset de déplacement.

`FenetreVisualisation`
: Hérite de `JFrame`. Gère le chargement du fichier PIF (argument ou JFileChooser), appelle le `Decompresseur` et construit l'interface d'affichage.

`FenetreConvertisseur`
: Hérite de `JFrame`. Gère le chargement de l'image, appelle le `Compresseur` pour l'analyse, et construit l'interface avec l'image, les trois onglets de tables et le bouton de conversion.

`PanneauImage`
: Hérite de `JPanel`. Panneau simple dédié à l'affichage de l'aperçu de l'image dans le Convertisseur.

3.2.4 Points d'entrée

`Convertisseur`
: Contient le `main` du programme de conversion. Se résume à créer une `FenetreConvertisseur` avec les arguments de la ligne de commande et à l'afficher.

`Visualisateur`
: Contient le `main` du programme de visualisation. Se résume à créer une `FenetreVisualisatio` avec les arguments de la ligne de commande et à l'afficher.

3.3 Nos choix techniques

3.3.1 Opérations binaires plutôt que Color()

Pour extraire les composantes RGB d'un pixel, on utilise des décalages de bits :

```
int r = (pixel » 16) & 0xFF;
int v = (pixel » 8) & 0xFF;
int b = pixel & 0xFF;
```

On aurait pu utiliser la classe `Color` de Java, mais cela crée un objet à chaque pixel, ce qui est coûteux pour une image de plusieurs millions de pixels. Les opérations binaires sont plus rapides et plus directes.

3.3.2 EcritureBinaire et LectureBinaire maison

On a développé nos propres classes pour l'écriture et la lecture bit par bit plutôt que d'utiliser `DataOutputStream` directement. Cela nous permet d'avoir un contrôle total sur le flux de bits et de gérer proprement le tampon de remplissage en fin de fichier.

3.3.3 Séparation analyser() et ecrire()

Dans le `Compresseur`, on a séparé l'analyse (calcul des fréquences et des codes) de l'écriture (production du fichier PIF). Cette décision est motivée par l'interface graphique : la `FenetreConvertisseur` a besoin d'accéder aux résultats intermédiaires de l'analyse pour afficher les tables, avant même que l'utilisateur décide de convertir.

3.3.4 Représentation des codes en String

La classe `Code` stocke les bits sous forme de `String`. On a envisagé une représentation en tableau de booléens ou en entier avec masque, mais la `String` est plus simple à déboguer et à afficher dans les tables de l'interface.

3.3.5 Convertisseur et Visualisateur hors MVC

Les deux classes contenant le `main` sont placées dans le package principal plutôt que dans `controleur`. Cela reflète leur rôle particulier : ce sont des points d'entrée du programme, pas des contrôleurs à proprement parler. Leurs `main` se limitent à trois lignes.

4. L'arbre binaire de Huffman

4.1 L'arbre de Huffman

L'arbre de Huffman est la structure centrale de notre projet. C'est lui qui permet d'associer à chaque valeur de composante (0 à 255) un code binaire dont la longueur est inversement proportionnelle à la fréquence de cette valeur dans l'image. Plus une valeur est fréquente, plus son code est court, et donc moins de bits sont nécessaires pour la représenter.

4.1.1 Les classes Noeud et Feuille

L'arbre est composé de deux types d'objets distincts : les noeuds internes, représentés par la classe `Noeud`, et les noeuds terminaux, représentés par la classe `Feuille` qui hérite de `Noeud`.

La classe `Noeud` représente un point de connexion dans l'arbre. Elle ne contient pas de valeur de couleur concrète : son rôle est uniquement structurel. Elle stocke une fréquence cumulée, qui correspond à la somme des fréquences de tous ses descendants, ainsi que deux références vers ses fils gauche et droit. Pour pouvoir être utilisée dans une `PriorityQueue` lors de la construction de l'arbre, `Noeud` implémente l'interface `Comparable`. La comparaison se fait sur la fréquence : un noeud moins fréquent est considéré comme "plus petit", ce qui garantit que la file de priorité extrait toujours les deux noeuds les moins fréquents en premier, conformément à l'algorithme de Huffman.

La classe `Feuille` est une sous-classe de `Noeud`. Elle représente l'extrémité d'une branche de l'arbre, c'est-à-dire une valeur de composante réelle présente dans l'image. En plus de la fréquence héritée, elle ajoute un attribut `valeur` de type `int` qui contient la valeur concrète du pixel pour cette composante (entre 0 et 255). Contrairement aux noeuds internes, une feuille n'a pas de fils. Lors du parcours de l'arbre pour décoder un flux de bits, on s'arrête dès qu'on rencontre une instance de `Feuille` : la valeur qu'elle contient est la valeur décodée.

Ce choix de modélisation par héritage simplifie le parcours de l'arbre. On n'a pas besoin de vérifier si les deux fils sont nuls pour savoir si on est sur une feuille : il suffit d'appeler la méthode `estUneFeuille()`, redéfinie dans `Feuille` pour retourner `true` et dans `Noeud` pour retourner `false`.

4.1.2 Construction de l'arbre

La classe `ArbreDeHuffman` ne stocke que la racine de l'arbre, le reste étant accessible récursivement depuis celle-ci. Sa construction se fait entièrement dans le constructeur, à partir d'une table de fréquences.

On commence par créer une `Feuille` pour chaque valeur dont la fréquence est non nulle, et on les insère toutes dans une `PriorityQueue`. Cette file de priorité les trie automatiquement par fréquence croissante grâce à l'implémentation de `Comparable`.

On répète ensuite l'opération suivante tant qu'il reste plus d'un élément dans la file : on extrait les deux noeuds de plus faible fréquence, on crée un `Noeud` parent dont la fréquence est égale à la somme des deux, et on réinsère ce parent dans la file. Le noeud extrait en premier (le moins fréquent des deux) devient le fils gauche, l'autre devient le fils droit.

Quand il ne reste plus qu'un seul élément dans la file, c'est la racine de l'arbre. Les valeurs les plus fréquentes se retrouvent alors naturellement proches de la racine, avec des chemins courts, tandis que les valeurs rares sont enfouies en profondeur, avec des chemins plus longs.

4.1.3 Lecture de l'arbre et génération des codes

Une fois l'arbre construit, on le parcourt récursivement depuis la racine pour générer les codes binaires. À chaque descente vers le fils gauche on ajoute un `0` au code en cours de construction, et à chaque descente vers le fils droit on ajoute un `1`. Quand on atteint une `Feuille`, le chemin parcouru depuis la racine constitue le code binaire de la valeur qu'elle contient.

Par exemple, sur l'arbre fourni dans le sujet, la valeur A possède le code `0` : elle est directement accessible en descendant une seule fois à gauche depuis la racine. La valeur B a le code `10` : on descend d'abord à droite, puis à gauche. La valeur C a le code `11` : deux descentes à droite. Ce sont bien les valeurs les plus fréquentes qui ont les codes les plus courts.

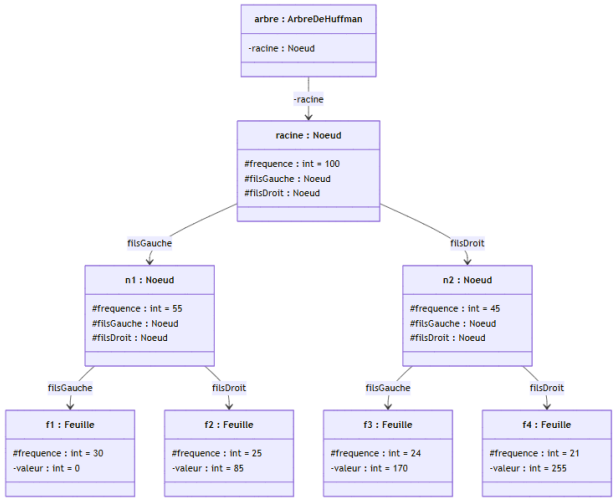


FIGURE 4.1 – Diagramme d'objets de l'arbre de Huffman, exemple tiré du sujet

5. Encodage — côté Convertisseur

5.1 Les tables de fréquences

La première étape de la compression est de parcourir tous les pixels de l'image et de compter combien de fois chaque valeur (0 à 255) apparaît pour chaque composante RGB. On obtient trois tables de 256 entrées.

Par exemple, pour une image donnée, la table du rouge pourrait contenir :

Valeur	Fréquence
0	207 152
1	1 480
2	0
...	...
255	346 944

5.2 Les codes initiaux de Huffman

Une fois l'arbre construit, on le parcourt en profondeur. À chaque descente vers un fils gauche on ajoute un `0`, vers un fils droit on ajoute un `1`. Quand on atteint une feuille, le chemin parcouru est le code de cette valeur.

Les valeurs les plus fréquentes se retrouvent plus hautes dans l'arbre, donc avec des codes plus courts. C'est ce qui permet de compresser.

Valeur	Code initial
0	0
85	101
170	100
255	11

5.3 Les codes canoniques

5.3.1 Pourquoi transformer les codes

Les codes initiaux dépendent de l'ordre exact dans lequel les noeuds ont été fusionnés lors de la construction de l'arbre. Pour que le décompresseur puisse retrouver les mêmes codes sans stocker l'arbre entier, on applique une transformation canonique.

5.3.2 Comment fonctionne la transformation

1. On trie les codes par longueur croissante, puis par valeur croissante pour les codes de même longueur.
2. Le premier code est rempli de zéros.
3. Chaque code suivant est le code précédent augmenté de 1, puis complété à droite avec des zéros pour atteindre la longueur voulue.

Valeur	Code initial	Code canonique
0	0	0
255	11	10
85	101	110
170	100	111

5.3.3 Ce qu'on stocke dans le fichier

Le bénéfice est que le décompresseur n'a besoin que de connaître la **longueur** de chaque code pour les recalculer exactement. On stocke donc 256 octets par composante : un octet par valeur (0 à 255) indiquant la longueur de son code. Un octet à 0 signifie que cette valeur est absente de l'image.

5.4 Le format du fichier PIF

Un fichier PIF est structuré en trois parties :

Section	Taille	Contenu
En-tête	4 octets	Largeur (2 octets) + Hauteur (2 octets)
Tables	768 octets	3 tables de 256 octets (R, V, B)
Pixels	Variable	Codes canoniques des pixels en continu

Les pixels sont écrits de gauche à droite, de haut en bas. Pour chaque pixel, on écrit le code du rouge, puis du vert, puis du bleu. Les bits s'enchaînent sans séparation, ce qui fait qu'un octet peut contenir des données de plusieurs pixels différents.

6. Décodage — côté Visualisateur

6.1 Lecture de l'en-tête

La décompression commence par la lecture des 4 premiers octets du fichier pour récupérer la largeur et la hauteur de l'image.

6.2 Reconstruction des codes canoniques

Pour chaque composante, on lit les 256 octets de longueurs. On applique ensuite exactement le même algorithme que lors de l'encodage pour recalculer les codes canoniques. Comme la procédure est déterministe, les mêmes longueurs produisent les mêmes codes, sans avoir besoin de stocker les codes eux-mêmes.

6.3 Reconstruction de l'arbre

Une fois les codes canoniques recalculés, on reconstruit l'arbre de Huffman en insérant chaque code valeur par valeur. Pour chaque bit du code, on descend à gauche (0) ou à droite (1) dans l'arbre, en créant les noeuds manquants. Quand on a épuisé tous les bits d'un code, on place une feuille avec la valeur correspondante.

6.4 Décodage des pixels

Pour décoder les pixels, on lit les bits du fichier un par un. À chaque bit, on descend dans l'arbre (gauche pour 0, droite pour 1). Quand on atteint une feuille, on a retrouvé la valeur de la composante. On repart alors de la racine pour décoder la composante suivante.

Ce processus se répète pour les trois composantes de chaque pixel, jusqu'à avoir reconstruit tous les pixels de l'image.

7. Conclusion

7.1 Conclusion de Kevin Yilmaz

Cette SAE a été une expérience particulièrement enrichissante pour moi. Je me suis principalement occupé de la partie modèle du projet, c'est-à-dire de toute la logique métier : la construction de l'arbre de Huffman, la génération des codes, la transformation canonique, ainsi que les classes d'écriture et de lecture binaire. C'est la partie la plus technique et la plus proche de l'algorithme pur, et c'est ce qui m'a le plus intéressé. J'ai également développé les contrôleurs `ControleurSouris` et `ControleurBouton`, qui font le lien entre les actions de l'utilisateur et la logique du programme. Ce projet m'a permis de mieux comprendre comment on passe d'un algorithme théorique, vu en cours, à une implémentation concrète et fonctionnelle. La compression de Huffman n'était pour moi qu'un concept abstrait au début de la SAE ; aujourd'hui je suis capable de l'expliquer et de le coder de zéro.

7.2 Conclusion de Adam Barhani

De mon côté, je me suis occupé de la partie interface graphique du projet, c'est-à-dire des fenêtres, des panneaux d'affichage, et de toute la logique de chargement et d'affichage des fichiers. Développer une interface qui s'adapte dynamiquement à la taille de l'image, qui centre correctement le contenu et qui permet de déplacer l'image à la souris m'a demandé beaucoup d'itérations et de tests. Cette SAE m'a appris à mieux structurer une application Java avec une architecture MVC, et à comprendre l'importance de bien séparer les responsabilités entre les différentes couches du programme. Travailler en binôme sur un projet de cette ampleur m'a également appris à mieux communiquer et à coordonner mon travail avec celui de mon partenaire, ce qui est une compétence aussi importante que le code lui-même.